Problem Statements

1) A hospital management system integrates patient data from multiple external labs. Each lab provides reports using different methods such as `getBloodReport()`, `fetchLabResults()`, and `retrieveDiagnostics()`, while the hospital system expects a unified method `getPatientReport()`. The developers plan to introduce a component that converts these different method calls into a single standard format without modifying existing lab classes. Identify and implement the appropriate design pattern and explain how it resolves interface mismatches.

2) An online video streaming platform loads high-resolution videos using a class with methods like `loadVideo(String name)` and `playVideo()`. Since loading videos is resource-intensive, the system currently delays playback due to repeated loading of the same content. Developers want to introduce an intermediary that controls access and ensures videos are loaded only when needed and reused when requested again. Identify and implement the suitable design pattern and explain how it improves performance.

3) A gaming application displays thousands of identical trees, rocks, and buildings across different locations in a virtual environment. Each object currently stores properties like texture, color, and type along with position coordinates, resulting in high memory usage. Developers want to reuse common properties and store only unique positional data separately. Identify and implement the design pattern that best fits this requirement and explain how it reduces memory consumption

4) An e-commerce platform requires customers to go through multiple steps such as `login()`, `selectProduct()`, `makePayment()`, and `trackOrder()`. Currently, users must call each method individually, making the process complex. Developers want to provide a single method `completePurchase()` that internally coordinates all these steps. Identify and implement the design pattern used and explain how it simplifies system interaction.